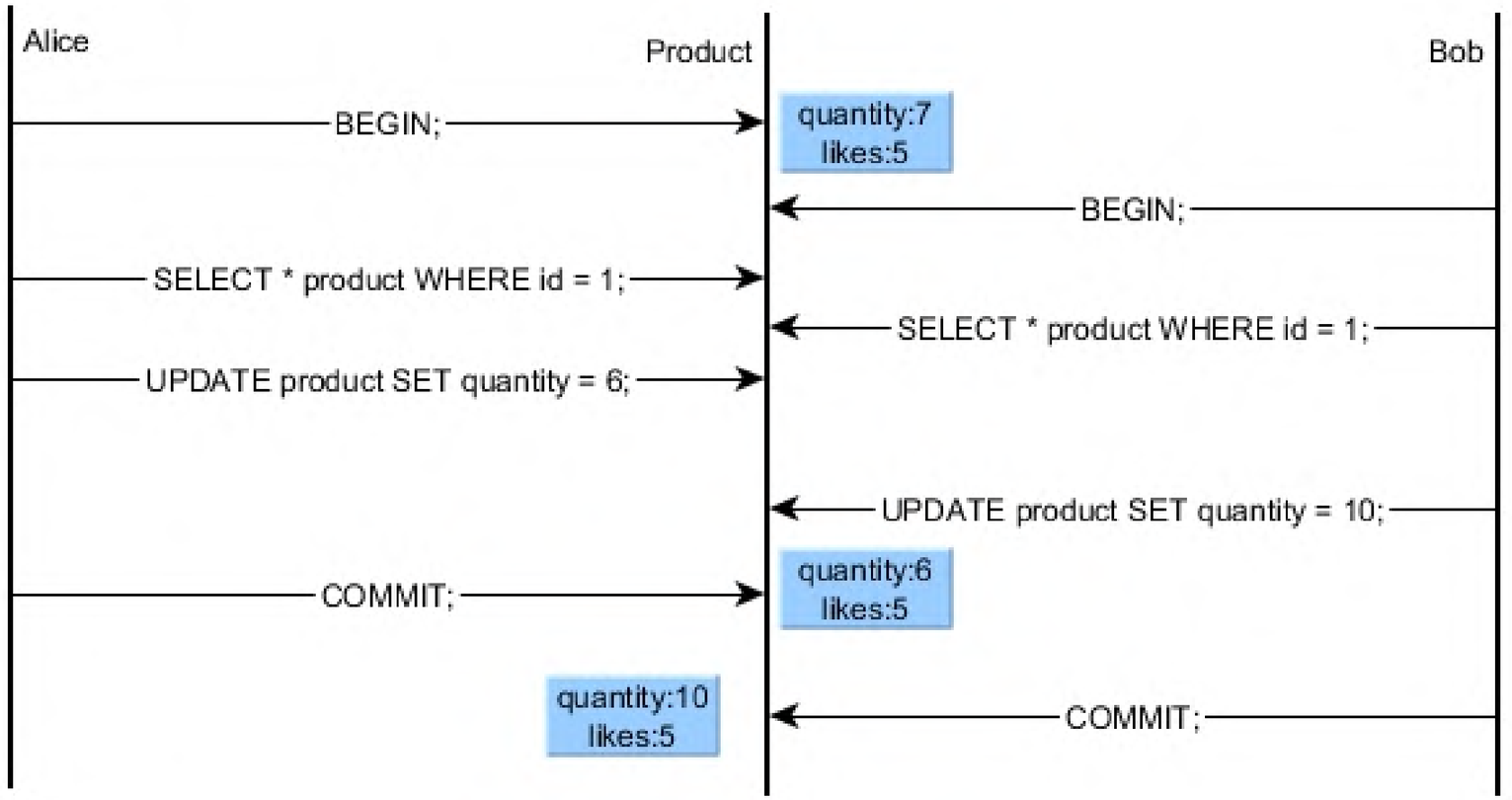
Database Concurrency



Lost Update

Lost update refers to a situation where the updates made by one transaction are overwritten or lost due to the concurrent execution of another conflicting transaction.

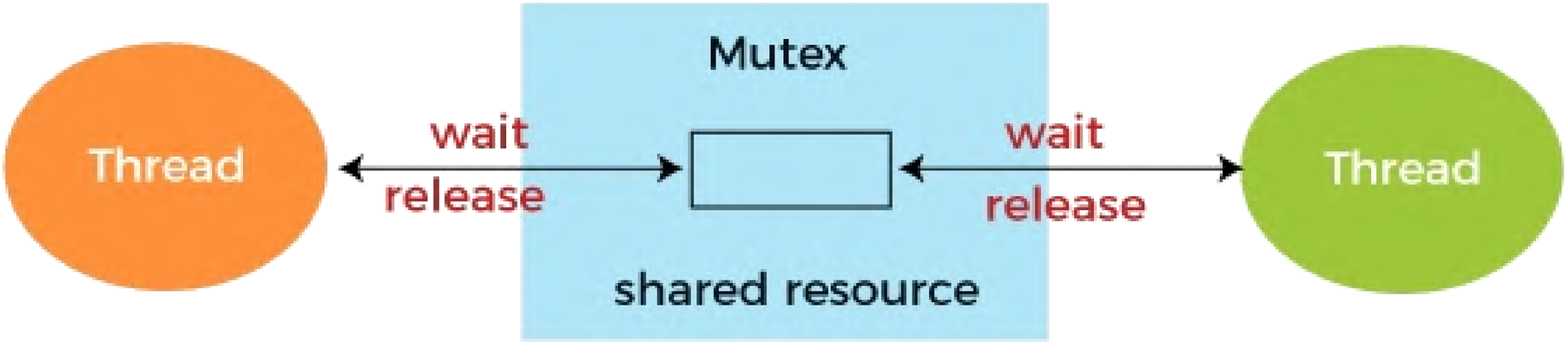


Synchronization Primitives

A synchronization primitive is a mechanism used to coordinate the execution of multiple threads or processes in order to prevent conflicts and ensure the correct and orderly execution of *shared resources*. C# synchronization primitives:

- `Monitor`
- `Mutex`
- `Semaphore`
- `ManualResetEvent`
- `AutoResetEvent`

Mutex

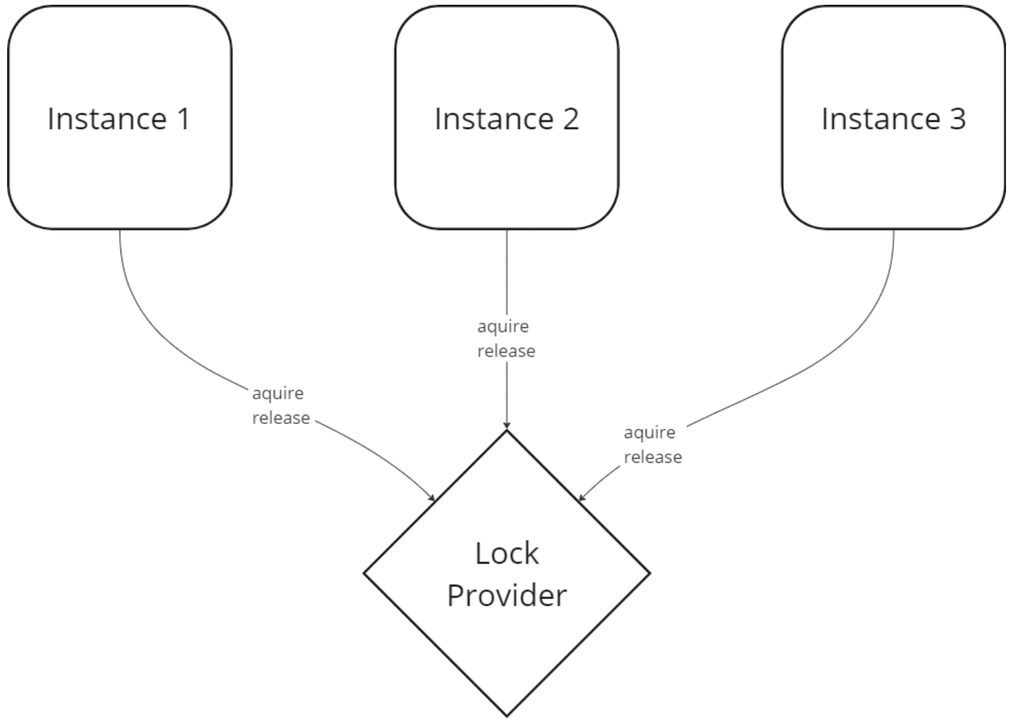

Mutex
Thread
wait
release
wait
release
Thread
shared resource

Distributed Lock

A distributed lock is a synchronization mechanism used in distributed systems to ensure exclusive access to a shared resource or a critical section of code. In a distributed system, where multiple processes or nodes operate concurrently, it is essential to coordinate their access to prevent conflicts and ensure data consistency.

DRIM

Distributed Lock



redis

amazon DynamoDB

ZooKeeper

etcd

SELECT FOR UPDATE

The SELECT FOR UPDATE statement is used in SQL to lock the selected rows in a table during a transaction. This ensures that other transactions cannot modify those rows until the current transaction is completed.

The syntax for SELECT FOR UPDATE varies depending on the specific database management system (DBMS) you are using. Here's a general example:

```
SELECT column1, column2, ...
FROM table
WHERE condition
FOR UPDATE;
```

Row Versioning

Database row versioning concurrency, also known as optimistic concurrency control or optimistic locking, is a technique used to handle concurrent modifications to database records. It allows multiple transactions to read and potentially update the same database row concurrently while minimizing conflicts and ensuring data integrity.

The basic idea behind row versioning concurrency is to use a version number or timestamp associated with each row in the database. When a transaction reads a row, it takes note of the version number or timestamp. When the transaction wants to update the row, it compares the version number or timestamp with the one it previously read. If the versions match, it proceeds with the update. However, if the versions differ, it means that another transaction has modified the row since the current transaction read it, and a conflict has occurred.

Optimistic vs Pessimistic Concurrency

- **Optimistic concurrency.** It assumes that conflicts between multiple threads or users accessing the same resource are infrequent. It allows concurrent operations to proceed without explicit locking or blocking, assuming that conflicts will be resolved later during the validation or commit phase.

- **Pessimistic Concurrency.** It assumes that conflicts between multiple threads or users accessing the same resource are likely to occur. It takes a more cautious approach by acquiring locks or other forms of synchronization before accessing the resource, ensuring exclusive access to the shared resource.

Transaction Isolation Levels

Transaction isolation levels are a concept in database systems that define how concurrent transactions interact with each other. These levels determine the degree to which one transaction can see changes made by other concurrent transactions.

1. Read Uncommitted
2. Read Committed
3. Repeatable Read
4. Snapshot Isolation
5. Serializable

Transaction Isolation Levels

It's important to note that not all database systems support all these isolation levels, and the actual implementation and behavior may vary.

Additionally, higher isolation levels often come with increased overhead and potential for reduced concurrency. Therefore, it's crucial to choose an appropriate isolation level based on the requirements of your application.

Repeatable Read

Repeatable Read isolation level is a concept used in database management systems to control the visibility and consistency of data during transactions. In this isolation level, a transaction sees a consistent snapshot of data as it existed at the beginning of the transaction and remains unaffected by any subsequent changes made by other transactions until the transaction completes.

Repeatable Read isolation level prevents lost updates.

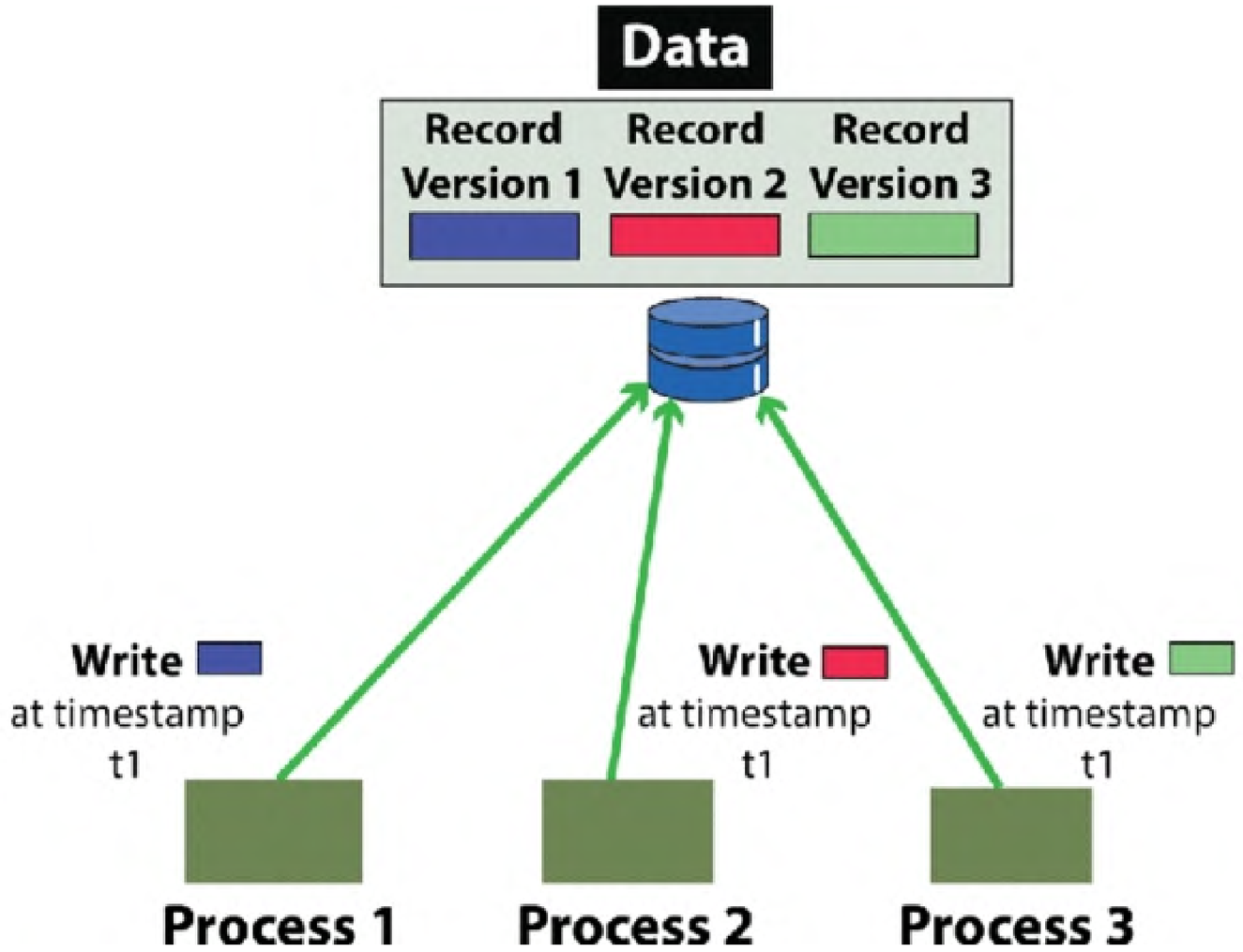
MVCC

Multi-Version Concurrency Control is a concurrency control technique used in database systems to manage concurrent transactions. It allows multiple transactions to access and modify data simultaneously while maintaining data consistency and isolation.

In traditional concurrency control methods like locking, transactions acquire locks on data items to prevent conflicts with other transactions. However, this approach can lead to contention and reduced concurrency, as transactions may need to wait for locks held by other transactions.

MVCC, on the other hand, avoids locking conflicts by creating multiple versions of data items and allowing transactions to operate on different versions concurrently. Each transaction works on a snapshot of the database at the time it started, providing a consistent view of the data.

MVCC


Data
Record Version 1
Record Version 2
Record Version 3
Write at timestamp t1
Write at timestamp t1
Write at timestamp t1
Process 1
Process 2
Process 3

DRIM

Thank you! Questions?

